EEG Brain Speller Project

Complete Requirements & Setup Guide for Mentees

Brain-Computer Interface (BCI) | Signal Processing | Machine Learning

---

1. Project Overview

You will build an EEG-based brain speller — a Brain-Computer Interface (BCI) system that allows a person to type characters using only their brain signals, with no physical movement. The system detects specific brainwave patterns in response to visual stimuli and uses machine learning to identify which letter the user intended.

The most common paradigm is the P300 Speller: a 6×6 matrix of characters flashes row by row and column by column. When the target character flashes, the brain produces a distinct EEG response called the P300 component (~300 ms after the flash). Your classifier detects this response to determine the intended character.

---

2. Environment Setup

2.1 Python Version

- Python 3.9 or higher (3.10 recommended)
- Use a virtual environment:

```
python -m venv eeg_env
source eeg_env/bin/activate   # Linux / macOS
eeg_env\Scripts\activate      # Windows
```

2.2 Core Python Libraries

Install all dependencies with:

```
pip install mne numpy scipy scikit-learn matplotlib seaborn moabb
pip install torch torchvision torchaudio   # for deep learning
pip install psychopy                        # for stimulus delivery
```

Library	Version	Purpose
mne	>=1.6	EEG loading, filtering, epoching, ICA

Library	Version	Purpose
numpy	>=1.24	Array operations and signal math
scipy	>=1.11	Bandpass filters, FFT, stats
scikit-learn	>=1.3	LDA, SVM, cross-validation, metrics
moabb	>=0.5	BCI dataset download and benchmarking
torch	>=2.0	Deep learning (EEGNet, CNN models)
matplotlib	>=3.7	Plotting ERP waveforms and results
seaborn	>=0.12	Heatmaps and confusion matrices
psychopy	>=2023.1	Visual stimulus delivery interface
pandas	>=2.0	Data management and result logging

2.3 Optional / Advanced

- **Deep learning toolkit built for EEG (wraps PyTorch)** braindecode
- **Riemannian geometry classifiers (strong baseline for EEG)** pyriemann
- **Automatic bad epoch rejection** autoreject

```
pip install braindecode pyriemann autoreject
```

---

3. Datasets

3.1 Primary Dataset (Start Here)

Use MOABB to download datasets programmatically. No manual downloading required.

```
from moabb.datasets import BNCI2014_009, EPFLP300
dataset = BNCI2014_009()   # P300 speller, 10 subjects
dataset.download()
```

3.2 Recommended Datasets by Paradigm

Dataset	Paradigm	Subjects	How to Access
BNCI2014-009 (EPFL P300)	P300 speller	10	moabb: BNCI2014_009()
BCI Competition III Dataset II	P300 speller	2	https://www.bbci.de/competition/iii/
Benchmark SSVEP (Wang 2017)	SSVEP	35	moabb: Wang2016()
BCI Competition IV 2a	Motor imagery	9	moabb: BNCI2014_001()

Dataset	Paradigm	Subjects	How to Access
PhysioNet EEG Motor Dataset	Motor imagery	109	mne.datasets.eegbci

For a P300 speller project, start with BNCI2014-009 or BCI Competition III Dataset II. Both have ground-truth labels for every character and multi-session recordings.

3.3 Loading Data with MNE

```
import mne
raw = mne.io.read_raw_fif('subject01.fif', preload=True)
print(raw.info)   # check sampling rate, channel names, duration
```

---

4. Signal Processing Pipeline

Implement each stage in order. Do not skip steps.

Stage 1 — Preprocessing

- Bandpass filter: 0.1–30 Hz (P300) or 1–45 Hz (SSVEP/MI)
  - raw.filter(l_freq=0.1, h_freq=30.0)
- Notch filter at 50 Hz (power line noise in India)
  - raw.notch_filter(freqs=50)
- Re-reference to average or linked mastoids
  - raw.set_eeg_reference('average')
- Bad channel interpolation (mark bad channels first)
- Artifact rejection: use ICA to remove eye blinks and muscle artifacts

Stage 2 — Epoching

- Extract epochs locked to stimulus onset events
- Epoch window: −200 ms to +800 ms around each flash (for P300)
- Apply baseline correction using the −200 to 0 ms pre-stimulus window

```
events, event_id = mne.events_from_annotations(raw)
epochs = mne.Epochs(raw, events, tmin=-0.2, tmax=0.8, baseline=(-0.2,
0))
```

Stage 3 — Feature Extraction

Paradigm	Feature Method	Notes
P300	Downsample epoch waveform	e.g. 240 samples → 30 features per channel
P300	Xdawn spatial filtering	Enhances SNR of ERP, strong baseline
SSVEP	FFT amplitude at stim freq	Check fundamental + 2nd harmonic
SSVEP	Canonical Correlation (CCA)	Compare EEG to reference sinusoids
Motor imagery	CSP (Common Spatial Patterns)	Classic ERD/ERS feature extractor
All	Covariance matrices (Riemannian)	Advanced: use pyriemann library

Stage 4 — Classification

- Baseline model: Linear Discriminant Analysis (LDA)
  - from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
- Strong classical baseline: SVM with RBF kernel
- Deep learning: EEGNet (compact CNN for EEG) — use braindecode or implement from the 2018 paper
- Ensemble: average classifier scores across multiple flash repetitions (improves accuracy significantly)

Stage 5 — Evaluation

- Use stratified k-fold cross-validation (k=5 or k=10)
- Report: accuracy, precision, recall, F1, and confusion matrix
- Primary metric: Information Transfer Rate (ITR) in bits/minute

ITR Formula:

```
ITR = (log2(N) + P*log2(P) + (1-P)*log2((1-P)/(N-1))) * 60 / T
```

Where N = number of symbols (e.g. 36), P = classification accuracy (0–1), T = trial duration in seconds.

---

5. Recommended Project Structure

```
eeg_speller/
  ├── data/              # raw and preprocessed EEG files
  ├── notebooks/         # Jupyter notebooks for exploration
  ├── src/
  │   ├── preprocess.py  # filtering, epoching, artifact removal
```

```
│   ├── features.py    # feature extraction functions
│   ├── models.py      # classifiers (LDA, SVM, EEGNet)
│   ├── evaluate.py    # cross-validation, ITR calculation
│   └── speller_ui.py  # optional: Psychopy stimulus interface
├── results/           # saved models, plots, metrics
├── requirements.txt
└── README.md
```

---

6. Suggested Milestones

Week	Milestone	Deliverable
1–2	Environment setup + data loading	Load dataset, plot raw EEG, identify channels
3–4	Preprocessing pipeline	Filtered + epoched data, ICA artifact removal
5–6	Feature extraction	Feature matrix ready for classifier input
7–8	Baseline classifier	LDA / SVM with cross-validation accuracy reported
9–10	Deep learning model	EEGNet trained, compare vs. LDA baseline
11–12	Evaluation + report	ITR calculated, confusion matrix, final write-up

---

7. Key References & Resources

Papers to Read

- Farwell & Donchin (1988) — original P300 speller paper (foundational)
- Lawhern et al. (2018) — EEGNet: A Compact Convolutional Neural Network for EEG-based BCIs
- Lotte et al. (2018) — A review of classification algorithms for EEG-based BCIs (survey)
- Wang et al. (2017) — A benchmark dataset for SSVEP-based BCIs

Online Resources

- MNE-Python tutorials: https://mne.tools/stable/auto_tutorials/
- MOABB documentation: https://moabb.neurotechx.com/
- BCI Competition datasets: https://www.bbci.de/competition/
- EEGNet GitHub: https://github.com/vlawhern/arl-eegmodels
- Braindecode library: https://braindecode.org/

8. Competition Evaluation Criteria

Criterion	Weight	What We Look For
Classification accuracy	30%	Subject-level accuracy on held-out test set
ITR (bits/minute)	30%	Speed + accuracy combined; higher is better
Code quality	20%	Modular, readable, reproducible, with README
Signal processing choices	10%	Justification of filter settings, epoch window, features
Presentation / write-up	10%	Clear explanation of pipeline and results

Questions? Reach your mentor. Good luck!